

Introduction to NLP

Natural Language Processing

Beginner Friendly

Hands-On Demo

spaCy · sklearn · HuggingFace

Mr. Shridhar Galande

Introduction to NLP — Part 1: Foundations, Part 2: Embeddings & Transformers

Agenda

Part 1

Foundations of NLP

Part 2

Models & Applications

Session Agenda — Part 1

Topic	What You'll Learn
Introduction to NLP	What NLP is, real-world uses, pipeline overview
NLP Pipeline & Language Understanding	Lexical, Syntactic, Semantic, Pragmatic analysis
Text Preprocessing Basics	Tokenization, stemming, lemmatization, noise removal
Text Representation Techniques	BoW, N-grams, TF-IDF, Word Embeddings
Demo 1 — Text Preprocessing	Full spaCy pipeline on reviews

What is NLP?

Teaching computers to understand human language

NLP (Natural Language Processing) is a branch of AI that helps machines read, understand, and generate text — just like a human would.



Google Search

Understands your query even with typos



Chatbots

ChatGPT, Claude, Gemini etc. understand & respond



Translation

Google Translate converts language instantly



Spam Filters

Gmail detects and blocks spam emails

💡 **Key Insight:** Computers see text as raw characters — NLP gives them the ability to understand meaning, intent, and context.

A simplified NLP Pipeline

Text goes through multiple stages before a machine can understand it

Input: "The movie was absolutely fantastic! I loved every scene."



Each step builds on the previous one — like a factory assembly line for language!

Lexical Analysis

POS Tagging & Morphology — understanding individual words

"What are the words and what form are they in?" -> The focus is on **tokenization**, **morphology**, and **word-level processing**.

What is it?

POS Tagging labels each word with its grammatical role:

Noun, Verb, Adjective, Adverb, etc.

Morphology studies word forms:

'run' → runs, running, ran

Sentence: "She runs fast in the park"

She → PRON (Pronoun)

runs → VERB

fast → ADV (Adverb)

in → ADP (Preposition)

the → DET (Determiner)

park → NOUN

Why does this matter?



Search Engines

Know that 'apple' is a noun (company or fruit?) from surrounding context



Grammar Check

Grammarly detects 'She run fast' is wrong because run must be VERB+3rd-person-s



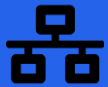
Chatbots

Extract 'book a flight' → VERB=book, NOUN=flight → triggers booking flow

Syntactic Analysis

Dependency Parsing · Parse Trees · Chunking

Focuses on grammatical structure by analyzing how words combine into phrases and sentences, with key tasks such as Constituency Parsing, which identifies nested phrase structures like Noun Phrases (NP), Verb Phrases (VP), and Prepositional Phrases (PP).



Dependency Parsing

Maps word-to-word grammatical relations

Info extraction · Relation detection · Semantic search



Parse Trees

Hierarchical phrase structure grammar

Grammar checking · MT parsing · Syntax education



Chunking

Flat phrase grouping & extraction

NER grouping · Search indexing · QA span finding

Dependency Parsing & Parse Trees



Dependency Parsing

"The dog chased the cat"

ROOT → chased

nsubj → dog

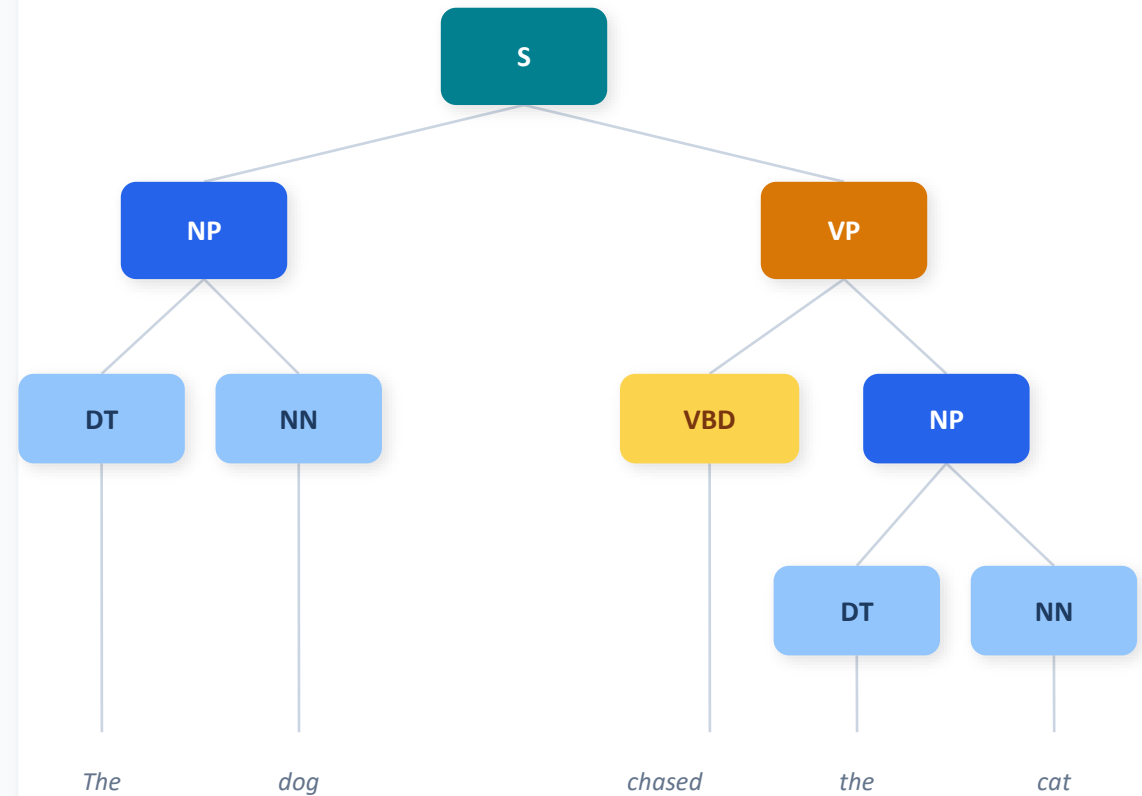
dobj → cat

det → The / the

Key: Arrows flow from head → dependent. Every word connects back to a single ROOT node.



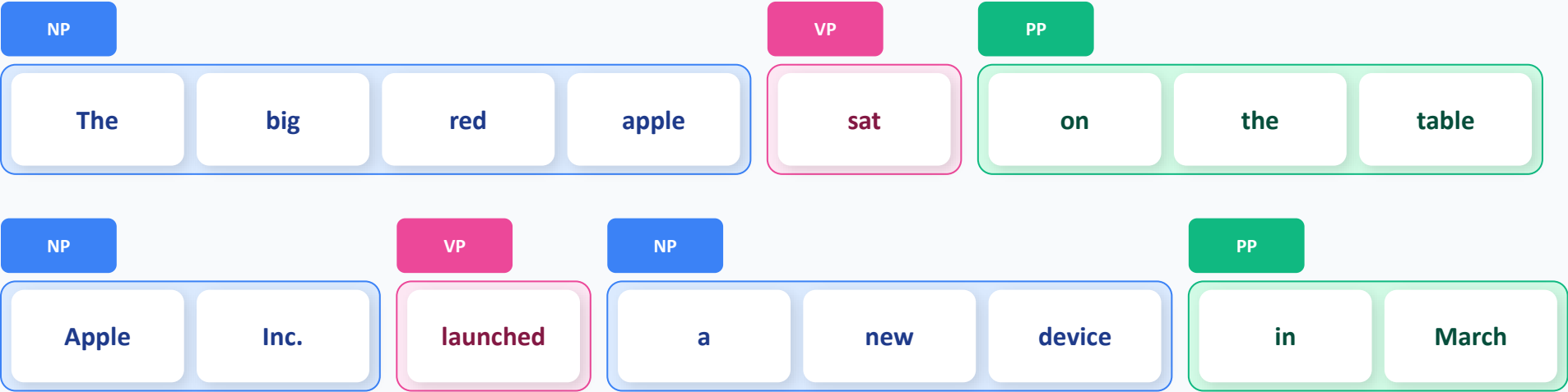
Parse Trees



Key: Phrases nest recursively — VP contains its own inner NP.

Chunking & Phrase Extraction

Chunking groups adjacent words into non-overlapping, non-recursive phrases (chunks). Unlike full parse trees, chunks are flat — faster and sufficient for most NLP pipelines.



Parse Tree vs. Chunking — key difference

	Parse Tree	Chunking
Depth	Fully recursive (VP → NP → DT + NN)	Flat, non-recursive brackets
Speed	Slower — full grammar required	Fast — regex / ML over POS tags
Use	MT, deep semantics, grammar checks	NER, search indexing, QA spans

Semantic & Pragmatic Analysis

Understanding meaning and real-world intent

NLP goes beyond grammar — it must understand what words mean (semantics) and what speakers intend (pragmatics). These two layers are essential for chatbots, sentiment analysis, machine translation, and dialogue systems.



Semantic Analysis

Meaning of words & sentences

- Word Sense Disambiguation
- Semantic Roles (Agent/Patient/Action)
- Context-dependent interpretation



Pragmatic Analysis

Intent behind language in context

- Sarcasm & irony detection
- Conversational implicature
- Speech acts & real-world requests

Semantic Analysis

Studies the **MEANING** of words and sentences — going beyond surface form to understand what is actually being communicated.



Word Sense Disambiguation (WSD)

"I went to the bank"

River bank

A slope of land beside a body of water

Financial bank

A financial institution for deposits

Context decides:

"fishing by the bank" → river; "deposit at the bank" → financial



Semantic Roles (Thematic Roles)

"John kicked the ball"

Agent

"John"

— Does the action

Patient

"ball"

— Receives the action

Action

"kicked"

— The event itself

Used in: machine translation, QA systems, event extraction

Pragmatic Analysis

Studies language in **REAL-WORLD CONTEXT** — the intent behind words. Goes beyond literal meaning to capture what the speaker actually means.



Sarcasm & Irony Detection

"Oh great, another Monday!"

Literal

"Oh great" = positive words

Intent

Actually expressing frustration / dread

Cues pragmatics uses:

Tone contrast · Hyperbole · Contextual incongruity · Prior discourse



Conversational Context & Speech Acts

"Can you pass the salt?"

Literal

A question about physical ability

Intent

A polite **REQUEST** to pass the salt

NOT

Asking if you are physically capable

Grice's Maxims — why pragmatics works:

Quantity · Quality · Relation · Manner

Lexical = Words → Syntactic = Structure → Semantic = Meaning → Pragmatic = Intent

Text Preprocessing Basics

Cleaning and preparing raw text before feeding it to any model

 **Analogy:** Before cooking, you wash vegetables, peel them, and chop them up. Preprocessing does the same for text — we clean and normalize it before the 'cooking' (model training) begins.

Sentence Segmentation

```
"Hello world. How are you?" → ["Hello world.", "How are you?"]
```

Tokenization

```
"I love NLP!" → ["I", "love", "NLP", "!"]
```

Noise Removal & Regex

```
"Visit http://nlp.ai !!! 😊" → "Visit"
```

Lowercasing

```
"Apple" and "apple" → same token "apple"
```

Stopword Removal

```
"the cat sat on the mat" → "cat sat mat"
```

Stemming

```
"running", "runs" → "run"  
"studies" → "studi"
```

Stemming vs Lemmatization

Both reduce words to their base form — but very differently

Why reduce words? "running", "runs", "ran" all mean the same thing. Reducing them to a base form helps machines treat them as one concept — critical for search, classification, and text mining.

Stemming (Rough & Fast)

Chops off word endings using simple rules. No dictionary — it just strips suffixes.

Output may NOT be a real word!

Examples (Porter Stemmer):

running	→	run	✓
studies	→	studi	⚠
better	→	better	✗ no change!
historical	→	histor	⚠

Best for: speed-critical tasks — search indexing, large corpora

Lemmatization (Smart & Accurate)

Uses a full vocabulary dictionary and grammar rules. Always returns a real, valid word.

Output is ALWAYS a valid dictionary word!

Examples (spaCy Lemmatizer):

running	→	run	✓
studies	→	study	✓
better	→	good	✓
historical	→	historical	✓

Best for: sentiment analysis, QA systems, chatbots — accuracy matters

Stemming vs Lemmatization

Quick comparison & when to use each

	Stemming	Lemmatization
Method	Suffix-stripping rules	Dictionary + grammar rules
Speed	Very fast	Slower (dictionary lookup)
Accuracy	Lower — may get nonsense	High — always real words
Language	Language-specific rules	Needs trained vocabulary
Output	"studi", "histor"	"study", "historical"
Use case	Search engines, indexing	NLU, sentiment, QA, chatbots

Choose Stemming when: millions of docs, speed is critical, slight inaccuracy is OK

Choose Lemmatization when: meaning matters — chatbots, NLU pipelines, QA

Text Representation — Bag of Words (BoW)

How do we turn words into numbers machines can process?

Core Idea: Count how many times each word appears in a document. Ignore word order — treat the document as a literal "bag" of words.

Step-by-step example

1 Doc 1: "cat sat on mat"

2 Doc 2: "cat ate the rat"

Build vocabulary (unique words):



BoW Vectors:

Doc1: [1 1 1 1 0 0 0]

Doc2: [1 0 0 0 1 1 1]

Strengths

- ✓ Simple and fast to compute
- ✓ Works well for short texts
- ✓ Great baseline for text classification
- ✓ Easy to implement in any language

Limitations

- ✗ Loses all word order — "not good" = "good"?
- ✗ Vocabulary size explodes with large corpora
- ✗ No concept of similar words ("happy" ≠ "joyful")
- ✗ Sparse vectors — mostly zeros

N-grams — Capturing Context

Groups of N consecutive words — preserving word order locally

The problem with BoW: "New York" gets split into "New" + "York" — losing the meaning. "not good" becomes just "good". N-grams fix this by grouping adjacent words.

Sentence: "I love NLP very much"

Unigrams (1-gram)

Same as BoW — individual words

"I"

"love"

"NLP"

"very"

"much"

Bigrams (2-gram)

Pairs of words — captures phrases

"I love"

"love NLP"

"NLP very"

"very much"

Trigrams (3-gram)

Triplets — richer context

"I love NLP"

"love NLP very"

"NLP very much"

Rule of thumb: N=1 for broad coverage · N=2 for phrases · N=3+ for specific patterns. Higher N = more context but sparser data.

TF-IDF — The Formula

Term Frequency × Inverse Document Frequency



Intuition: A word is important if it appears **OFTEN** in **THIS** document, but **RARELY** across **ALL** documents. "the" appears everywhere → low importance. "photosynthesis" is rare → high importance in biology docs.

TF — Term Frequency

How often does the word appear in **THIS** document?

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in doc } d}{\text{Total number of words in doc } d}$$

IDF — Inverse Document Frequency

How **RARE** is this word across **ALL** documents?

$$IDF(t) = \log \left(\frac{\text{Total number of documents}}{\text{Documents containing term } t} \right)$$

$$TF - IDF(t, d) = TF(t, d) \times IDF(t)$$

TF-IDF — Worked Example

Seeing the formula in action with a film review document

Document: "The movie had brilliant cinematography and the acting was superb." | Corpus: 1000 movie reviews

Word	TF (in doc)	IDF (across docs)	TF-IDF score	Importance
the	0.12	0.01 (very common)	0.001	Low
movie	0.05	1.20 (somewhat rare)	0.060	Medium
cinematography	0.02	3.80 (very rare)	0.076	High
acting	0.04	1.50 (somewhat rare)	0.060	Medium
superb	0.01	2.90 (rare)	0.029	Medium

Key insight: "the" has high TF (appears often) but near-zero IDF (appears in ALL docs) → TF-IDF ≈ 0. "cinematography" has low TF but high IDF → TF-IDF is highest. TF-IDF automatically finds the **DISTINCTIVE** words.

Word Embeddings — Intuition

Representing words as vectors in a meaningful space

Problem with BoW / TF-IDF

- X "king" and "queen" treated as completely unrelated
- X "happy" and "joyful" look totally different to the model
- X No understanding of word similarity
- X Cannot handle unseen words ("OOV problem")

Word Embeddings Solution

Each word is mapped to a dense vector of numbers. Similar words end up CLOSE in vector space!

king → [0.2, 0.8, 0.1, ...]

queen → [0.2, 0.7, 0.2, ...]

happy → [0.9, 0.1, 0.7, ...]

joyful → [0.8, 0.1, 0.7, ...] ← very close!

Famous Word2Vec result: "king" – "man" + "woman" ≈ "queen"

Word2Vec

Predicts surrounding context words (CBOW) or target from context (Skip-gram). Most famous embedding model.

Google · 2013

GloVe

Uses global word co-occurrence statistics from large corpora. Combines local context with global matrix factorization.

Stanford · 2014

FastText

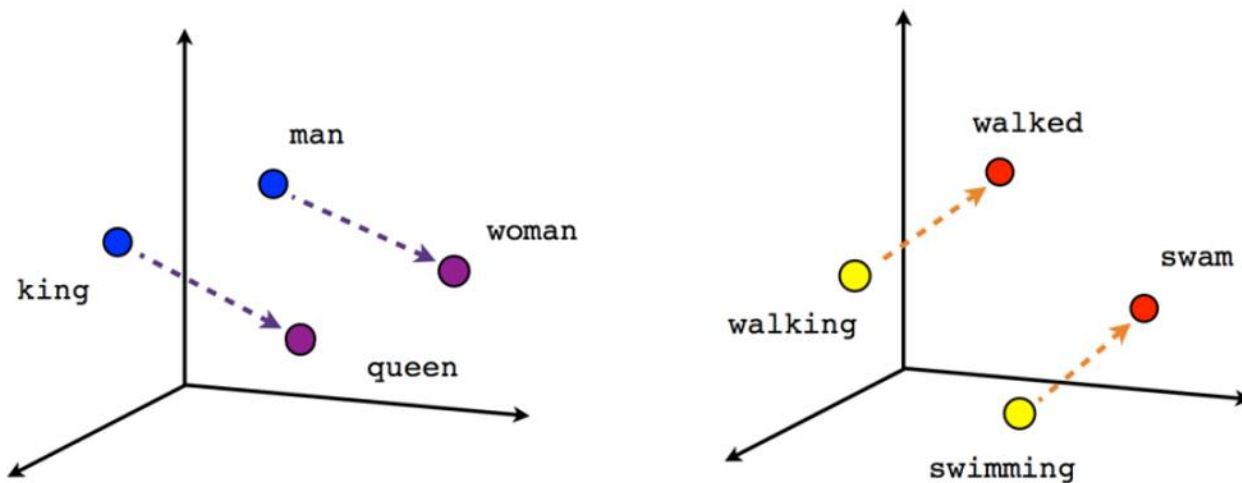
Breaks words into character n-grams. Handles unseen words, misspellings, and morphologically rich languages.

Facebook · 2016

Word Embeddings — Vector Space

How words cluster by meaning in high-dimensional space

Semantic clusters in vector space



Male-Female

Verb tense

Key properties

Similarity = distance

Cosine similarity measures how close two word vectors are

Arithmetic works

"king" - "man" + "woman" = "queen" (vector math!)

Dimensions = features

Each dimension may encode gender, age, royalty, etc.

Typically 100–300 dims

Dense vectors vs sparse BoW vectors of 100,000+

Transfer learning

Pre-trained on billions of words, reuse in your task



Demo 1 — Pre-processing + Linguistic Analysis

IMDb Movie Reviews · spaCy Pipeline



Load Review Data



Sentence
Segment



Tokenize



Regex
Clean



POS
Tag



Dependency
Parse



Chunk



Lemmatize

Input: "The Shawshank Redemption was an absolutely breathtaking film!" → Output: Tokens + POS Tags + Chunks + Lemmas

Break Time!

10 minutes

PART 2

Embeddings & Transformers

From Word Vectors to Modern Language Models — Architecture, Math & Applications

Word2Vec · GloVe · FastText

Self-Attention · Multi-Head

BERT · GPT · Applications

Mr. Shridhar Galande

Introduction to NLP — Part 2

Four building blocks — from word meaning to real-world transformer applications.

1

Embeddings

Word2Vec, GloVe, FastText & contextual representations

2

Transformer Architecture

Attention, multi-head attention & the full model

3

Applications Demo

Live notebook across encoder, decoder & seq2seq tasks

4

Wrap-Up

Recap, resources & open discussion

Why Bag-of-Words Isn't Enough

TF-IDF and BoW gave us numbers. They didn't give us meaning.

TF-IDF / Bag-of-Words

- Every word is an isolated dimension — no notion of meaning
- "excellent" and "great" are as unrelated as "excellent" and "chair"
- Vectors are sparse and grow with vocabulary size
- Cannot represent a word it has never seen before

What Embeddings Give Us

- Dense, fixed-size vectors (e.g. 300 numbers per word)
- "excellent" and "great" land close together in vector space
- Similarity becomes a distance we can measure
- The foundation every Transformer is built on

What Are Word Embeddings?

Representing meaning as points in space, not just counts.

An embedding maps every word to a dense vector of real numbers, learned so that words used in similar contexts end up with similar vectors.

Running Example — Customer Reviews

R1: The camera quality is excellent and photos look stunning in low light.

R2: Battery life is terrible, it barely lasts half a day.

R3: The waiter was friendly but the pasta arrived cold and bland.

The word "excellent" as a vector



Each dimension captures a latent aspect of meaning learned from data — no single one is human-labeled.

One-Hot vs. Dense Embeddings

Two ways to turn a word into numbers.

One-Hot Encoding

- Vector length = vocabulary size (often 20,000+)
- Exactly one 1, everywhere else 0
- Every word is equally distant from every other word
- No shared structure between related words

`excellent` = `[0,0,1,0,0,...,0]`

Dense Embedding

- Vector length is fixed and small (e.g. 100–300)
- Every dimension carries real-valued information
- Similar words → similar vectors → close in space
- Generalizes across the whole vocabulary

`excellent` = `[0.81, -0.22, 0.55, 0.09, ...]`

You Shall Know a Word by the Company It Keeps

The distributional hypothesis behind every modern embedding model.

Words that appear in similar contexts tend to have similar meaning. Word2Vec turns that idea into a training signal.

quality

is

excellent

and

photos

Context window (window size = 2) around the target word "excellent", drawn from R1 of our review dataset.

What the model learns

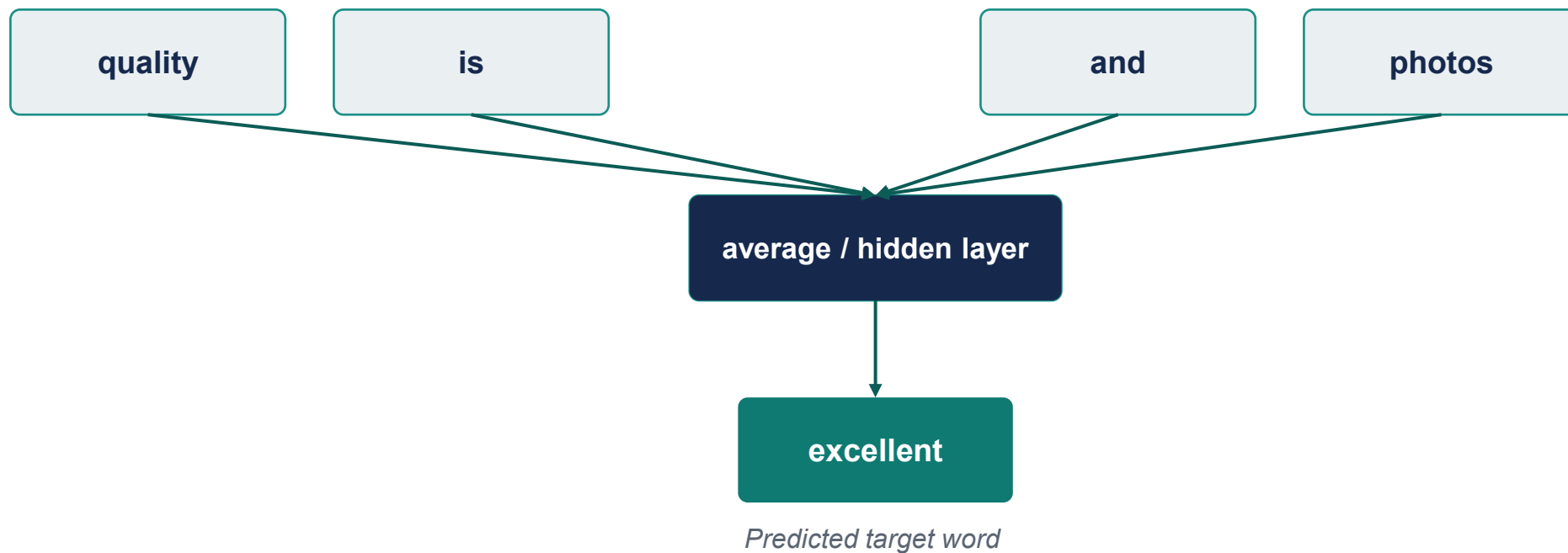
- Words like "excellent", "stunning", and "great" recur near positive product descriptions across many reviews
- Over millions of such windows, the model pulls their vectors closer together in space

CBOW — Continuous Bag of Words

Predict the missing target word from the words around it.

Task

- Given "quality is ____ and photos", predict "excellent"

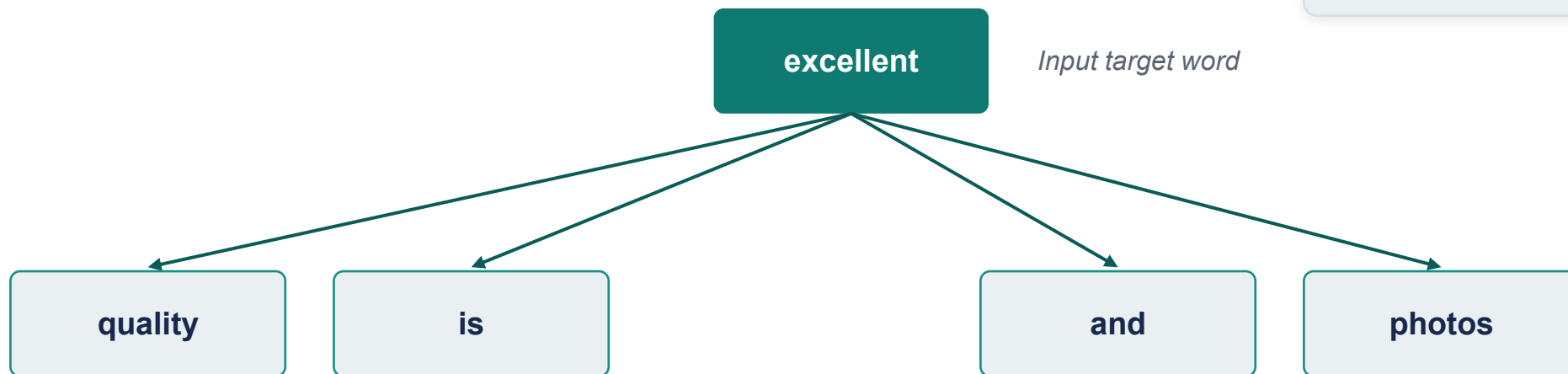


Skip-Gram — Predict the Context

Given the target word, predict each surrounding context word.

Task

- Given "excellent", predict "quality", "is", "and", "photos"



CBOW vs. Skip-Gram

Same model family, two different prediction directions.

Direction	Context → Target	Target → Context
Training speed	Faster (fewer predictions per window)	Slower (multiple predictions per word)
Best for	Frequent words, larger corpora	Rare words, smaller datasets
Typical use	Quick baseline embeddings	Higher-quality rare-word vectors

Global Vectors for Word Representation

Learning from how often word pairs co-occur across the whole corpus.

Instead of sliding a local window one sentence at a time, GloVe first builds a global word-word co-occurrence count, then factorizes it into dense vectors.

Co-occurrence counts (toy example)

	excellent	great	quality	bland
excellent	—	6	8	0
great	6	—	5	1
quality	8	5	—	1
bland	0	1	1	—

Why it works

- Captures corpus-wide statistics, not just local windows
- Ratios of co-occurrence probabilities encode meaningful relationships
- Often trains faster than pure prediction-based methods

COMPARISON

Word2Vec vs. GloVe

Two roads to the same destination: dense, meaningful word vectors.

	Word2Vec	GloVe
Approach	Predictive (neural network)	Count-based (matrix factorization)
Signal used	Local context windows	Global co-occurrence statistics
Training data view	One window at a time	Whole corpus statistics up front
Practical quality	Excellent, especially with negative sampling	Comparable — strong on analogy tasks

Subword Embeddings

What happens when a word has never been seen before?

FastText represents each word as a bag of character n-grams, so it can build a vector for words it has never seen — typos, rare terms, new coinages.

"excellent" → character 3-grams

exc

xce

cel

ell

lle

len

ent

Why it matters

- A misspelled review word like "excelent" still shares most of its n-grams with "excellent" — FastText still places it nearby
- Especially valuable for morphologically rich languages and informal, typo-heavy text like customer reviews

Embedding Arithmetic & Analogies

Vector directions capture relationships, not just similarity.

The classic example

- "king" – "man" + "woman" $\approx$ "queen"

In our review domain

- "excellent" – "positive" + "negative" $\approx$ "terrible"

Why this works

The training process arranges vector space so that consistent semantic relationships — gender, sentiment polarity, tense — become consistent directions. Adding or subtracting a direction moves you from one concept to a related one.

Measuring Similarity — Cosine Similarity

How close are two word vectors, regardless of their length?

$$\cos(\theta) = \frac{(A \cdot B)}{\|A\| \|B\|}$$

Reading the score

- 1.0 = pointing the same direction (very similar)
- 0.0 = unrelated · -1.0 = opposite meaning

Worked example — from our reviews

"excellent" vs "great"

0.87

"excellent" vs "bland"

0.09

Limitations of Static Embeddings

One vector per word — even when the word means different things.

The "cold" problem

- R3: "the pasta arrived cold" — temperature
- A different review: "the staff felt cold and distant" — personality
- Word2Vec, GloVe, and FastText assign both uses the exact same vector

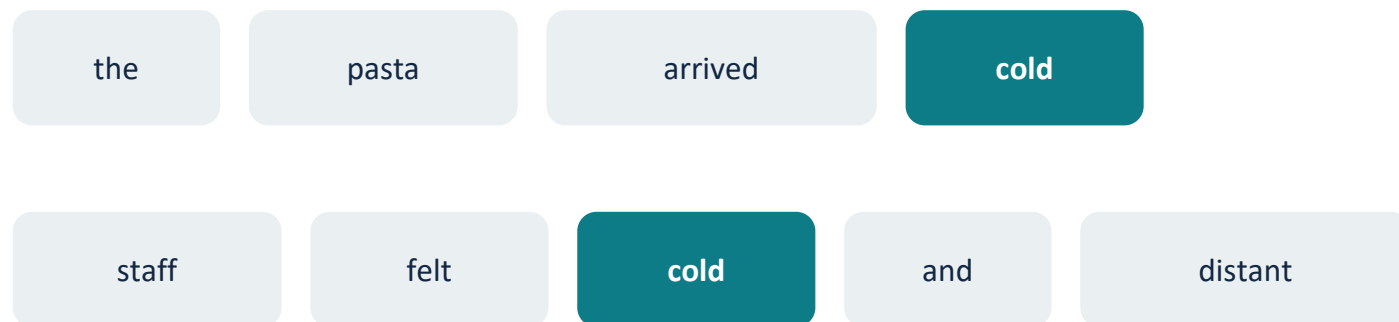
The core limitation

- Static embeddings are looked up once per word, independent of sentence
- No way to let surrounding context reshape the meaning
- This is exactly the gap contextual models were built to close

From Static to Contextual Embeddings

Letting the sentence decide what a word means.

ELMo (2018) was an early step: a bidirectional LSTM reads the full sentence and produces a different vector for the same word depending on context.



Same word "cold" → two different contextual vectors, because the surrounding words differ.

Where this leads

- Transformers (next section) generalize this idea with self-attention — every word's representation is computed fresh, using the whole sentence

Sentence & Document Embeddings

Combining word vectors into a single representation.

Simple pooling

- Mean pooling: average every word vector in the review
- Max pooling: take the strongest signal per dimension
- Fast and simple, but loses word order entirely

Learned sentence encoders

- Doc2Vec: extends Word2Vec with a document-level vector
- Sentence-BERT: fine-tunes a Transformer for sentence similarity
- Covered in depth in the Transformer Architecture section next

TOKENIZATION

Byte-Pair Encoding & Subword Tokenization

The bridge from word embeddings into every modern Transformer.

Instead of one token per whole word, BPE / WordPiece iteratively merge the most frequent character pairs, building a vocabulary of common subword pieces.

"unbelievably" → subword tokens

un

believ

ably

Why Transformers use it

- Fixed, manageable vocabulary size, no matter the language
- Handles rare and unseen words the same way FastText's n-grams do
- The direct predecessor to the tokenization used in BERT, GPT, and every model in the next section

Popular Embedding Tools & Libraries

What you'll actually reach for in practice.

1

Gensim

Train Word2Vec, GloVe & FastText models from scratch

2

spaCy

Ready-to-use pretrained word vectors, tightly integrated into NLP pipelines

3

HuggingFace

Contextual embeddings from pretrained Transformer models

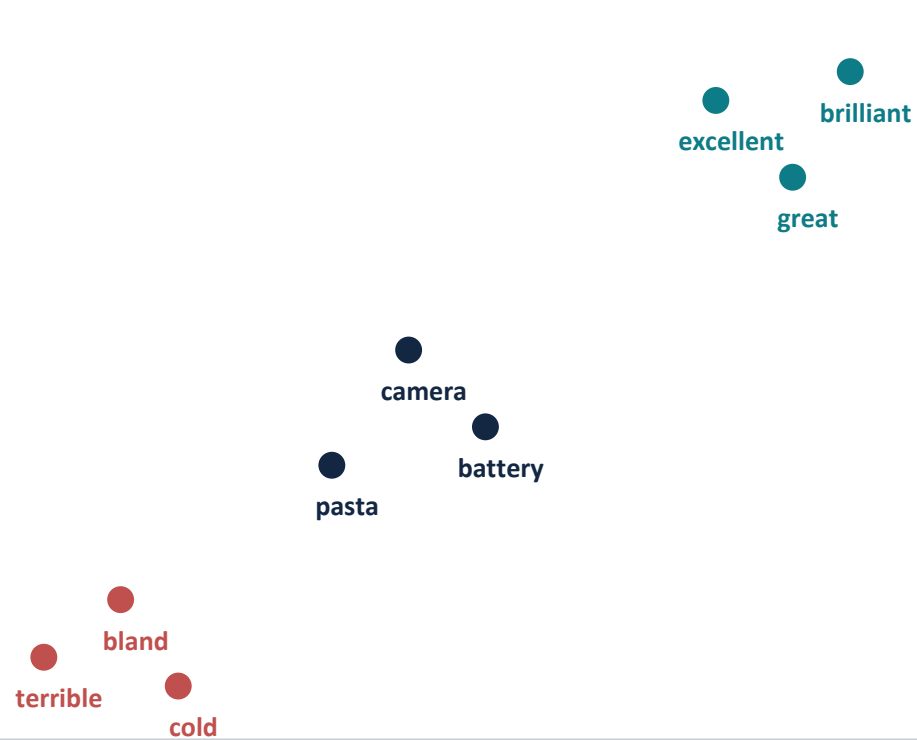
4

scikit-learn

TF-IDF baselines and classical vectorizers for comparison

Visualizing Embeddings

Projecting high-dimensional vectors down to 2D with t-SNE or PCA.

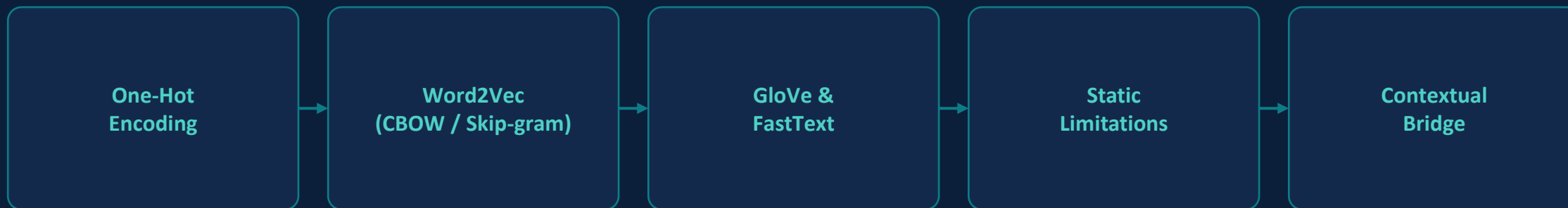


What to notice

- Positive-sentiment words cluster together
- Negative-sentiment words form their own cluster
- Product-noun words group separately from sentiment words
- Distance in this plot reflects distance in the real high-dimensional space

SECTION RECAP

The Embeddings Journey



Next: the architecture that made static, one-vector-per-word representations obsolete — Transformers.

Why Do We Need Transformers?

The Old Way: Reading One Word at a Time

RNNs process a sentence step by step, like reading with your finger under each word. Each word must wait for the one before it.

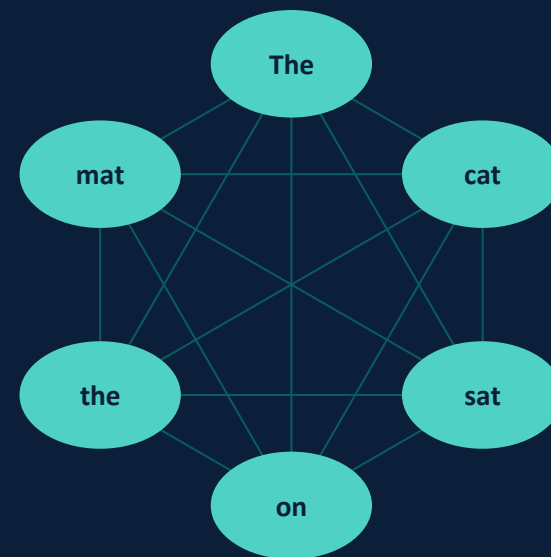


Slow, sequential — and by word 6, earlier context starts to fade.

- No parallel computation — must wait for each step
- Long sentences lose track of early context
- Training is slow on long sequences

The New Way: Reading All Words at Once

Transformers look at every word simultaneously and let attention decide which words matter to each other.



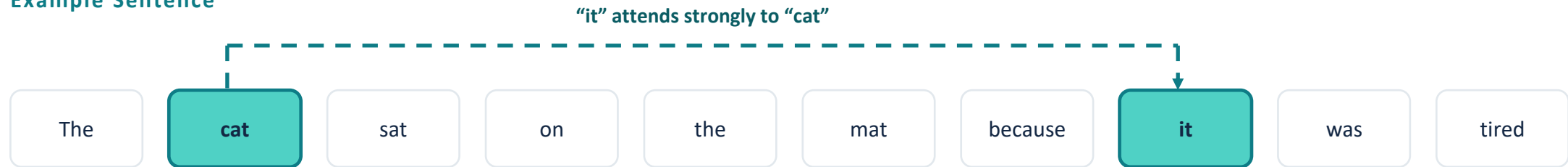
All words connect directly — no waiting in line.

ONE SIMPLE TRICK

The Core Idea: Attention

Every word can directly ask: “Which other words matter to me?”

Example Sentence



Self-attention lets the model figure out that “it” refers to “cat”, not “mat” — by directly comparing every word against every other word.

- No recurrence needed — Every word reaches every other word in a single step
- Fully parallel — All comparisons happen at the same time, not in sequence

Key Idea

Context comes from relationships between words — not their order in a queue.

The Building Blocks

Six simple ideas combine to form the full Transformer. We'll walk through each one.



1

Tokenization & Embeddings

Turn tokens into number vectors



2

Positional Encoding

Tell the model the word order



3

Self-Attention

Words look at other words



4

Multi-Head Attention

Several views at once



5

Feed-Forward Network

Each word “thinks” on its own



6

Residuals & Normalization

Keeps deep stacks stable

Turning Words into Numbers

Models can only do math, so every word is first split into tokens, then converted into a list of numbers (a vector) that captures its meaning.

Example: "I love pizza"

I



```
0.12  
-0.44  
0.81  
0.03
```

love



```
0.77  
0.29  
-0.18  
0.55
```

pizza



```
-0.31  
0.62  
0.44  
-0.09
```

Embedding vectors

Words with similar meanings end up with similar vectors — this is how the model represents meaning mathematically.

Teaching the Model Word Order

Self-attention alone has no sense of order — it would treat a sentence like a bag of words. Positional encoding adds “position” information into each embedding.

Order Changes Meaning

“Dog bites man” — ordinary news

“Man bites dog” — that's the headline!

Now the model knows both what each word means and where it sits in the sentence.



Each position gets its own unique numeric pattern, so “word 1” and “word 8” never look identical.

How Position Gets Added In

Word Embedding

meaning of “cat”

+

Positional Encoding

“position 2 in sentence”



Final Input Vector

meaning + position, combined

Self-Attention: The Library Analogy

Every word plays three roles at once. Think of it like searching a library:



Query

What am I looking for?

“Who is this action about?”



Key

What labels does each book have?

Each word's searchable “tag”



V

Value

What's actually inside the book?

Each word's real content

How it works

Every word turns into a Query, a Key, and a Value. The model compares each word's Query against every word's Key to get a relevance score, then blends the Values together, weighted by those scores.

The Attention Formula

$$\textit{Attention}(Q, K, V) = \textit{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

1 QK^T — Similarity Scores

Multiply each word's Query vector by every word's Key vector (dot product). This produces a raw relevance score between every pair of words in the sentence.

2 $\div \sqrt{d_k}$ — Scaling

Divide every score by the square root of the key dimension (d_k). Without this, large dot products push softmax into regions with tiny gradients, slowing learning.

3 $\textit{softmax}()$ — Normalize

Convert the scaled scores into probabilities that sum to 1 across each row. These are the final attention weights — how much focus each word gets.

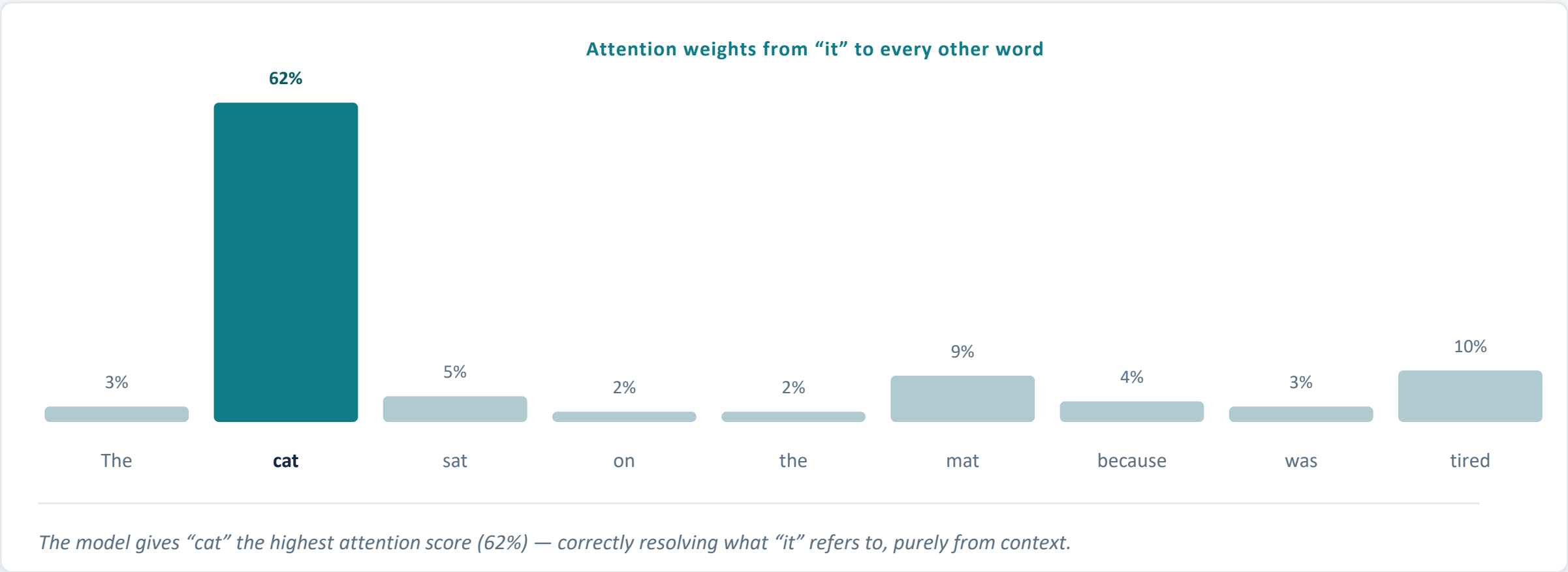
4 $\times V$ — Weighted Sum

Multiply each attention weight by its corresponding Value vector and sum them up. The result is a new, context-aware representation for each word.

d_k = dimension of the Key vectors (commonly 64 per attention head) — scaling keeps training stable as this dimension grows.

Self-Attention in Action

Sentence: “The cat sat on the mat because it was tired” — focus word: “it”



A Real Numeric Example

Sentence: “I love AI” — let's compute real Query, Key, and Value matrices by hand.

Step 0 — Word Embeddings (X)

Each word is already a 4-number vector (from Building Block 1).

I	1	0	1	0
love	0	2	0	2
AI	1	1	1	1

Learned Weight Matrices

During training, the model learns three weight matrices that project embeddings into Query, Key, and Value space. Here are simple example values for each.

W_Q	W_K	W_V
101	110	100
011	010	110
110	101	001
001	100	011

4x4 embeddings × 4x3 weights = 3x3 Q, K, V matrices (one row per word).

Computing Query, Key & Value

$Q = X \times W_Q \quad K = X \times W_K \quad V = X \times W_V$

Q — Query

I	2	1	1
love	0	2	4
AI	2	2	3

“What am I looking for?”

e.g. row “love” = [0, 1, 0] · W_Q = [0, 2, 4]

K — Key

I	2	1	1
love	2	2	0
AI	3	2	1

“What do I offer?”

e.g. row “love” × W_K = [2, 2, 0]

V — Value

I	1	0	1
love	2	4	2
AI	2	2	2

“What’s my actual content?”

e.g. row “love” × W_V = [2, 4, 2]

Similarity Scores: $Q \times K^T$

Every word's Query is compared against every word's Key using a dot product — producing a 3×3 score matrix.

Worked Example: “AI” attending to “AI”

$$Q_{AI} = [2, 2, 3] \quad K_{AI} = [3, 2, 1]$$

$$\text{score} = (2 \times 3) + (2 \times 2) + (3 \times 1)$$

$$= 6 + 4 + 3 = 13$$

Full Score Matrix ($Q \times K^T$)

	I	love	AI
I	6	6	9
love	6	4	8
AI	9	8	13

Notice: “AI” scores highest with itself (13) — and fairly high with “love” (8), since both carry related meaning in this toy example. Raw scores aren't bounded yet; that's what scaling and softmax fix next.

Scaling & Softmax

Divide by $\sqrt{d_k} = \sqrt{3} \approx 1.73$, then apply softmax to each row so weights sum to 1.

Raw Scores

I	6	6	9
love	6	4	8
AI	9	8	13

$\div \sqrt{3}$

Scaled Scores

I	3.46	3.46	5.20
love	3.46	2.31	4.62
AI	5.20	4.62	7.51

Attention Weights (after softmax)

Each row now sums to 1.00 — these are the final attention weights.

	I	love	AI
I	13%	13%	74%
love	22%	7%	71%
AI	9%	5%	86%

Darker cells = stronger attention. Both “I” and “AI” end up focusing most on “AI” — it carries the most distinctive meaning in this sentence.

The Final Output: Weighted Sum

Multiply the attention weights by V and sum — producing a new, context-aware vector for every word.

Worked Example: new vector for “AI”

weights = [0.09, 0.05, 0.87]

$$\begin{aligned} \text{output} &= 0.09 \times V_I + 0.05 \times V_{\text{love}} + 0.87 \times V_{\text{AI}} \\ &= 0.09[1,0,1] + 0.05[2,4,2] + 0.87[2,2,2] \\ &\approx [1.87, 1.92, 1.87] \end{aligned}$$

Final Context-Aware Vectors

I	1.87	2.00	1.87
love	1.78	1.69	1.78
AI	1.91	1.92	1.91

This is the payoff: each word's original embedding has been replaced with a new vector that blends in context from every other word — weighted by relevance. This is what flows into the next layer.

The Full Pipeline, Start to Finish

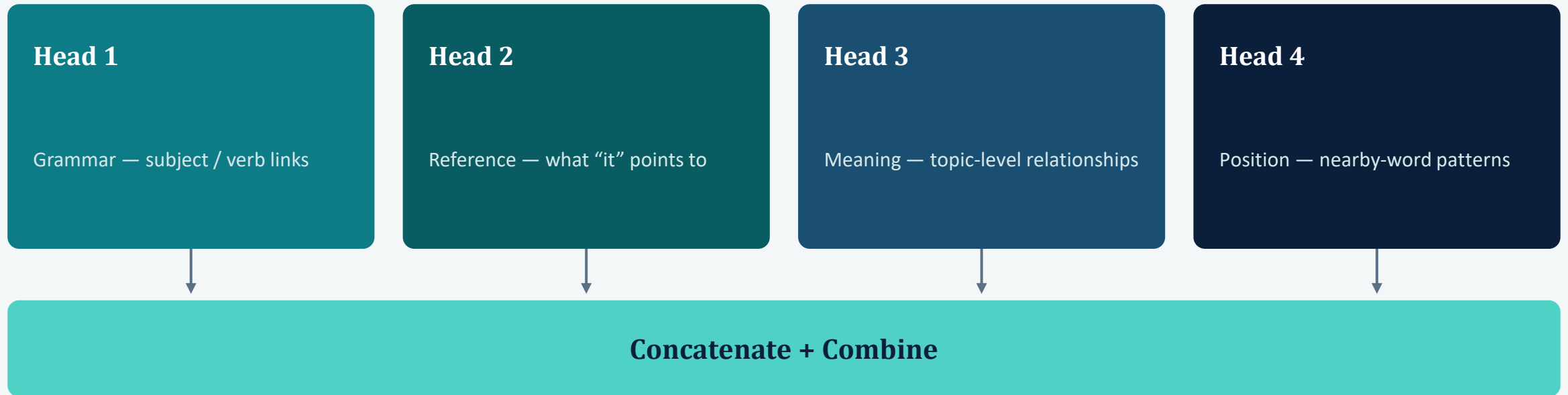


Same Math, Every Layer

- This exact sequence — project, score, scale, softmax, weighted sum — runs inside every attention head, in every layer, for every word, simultaneously.
- Multi-head attention just repeats this whole pipeline several times in parallel with different learned weight matrices, then combines the results.
- In this simplified attention calculation, WQ , WK and WV are the learned projection matrices.

Multi-Head Attention: Many Experts, One Sentence

One attention pass can only focus on one kind of relationship. So the model runs several attention “heads” in parallel, each specializing in something different.



Just like asking four different experts to read the same sentence and combining their notes, multi-head attention captures richer context than a single pass could.

Types of Attention

“Attention” isn't one single thing — it's a family of variants that differ in what attends to what, and how it's computed.



Self-Attention

WHAT attends to WHAT

Words attend to other words in the same sequence. Q, K, and V all come from the same input.

Every encoder layer; decoder's first attention step



Cross-Attention

WHAT attends to WHAT

Words in one sequence attend to words in a different sequence. Q comes from one side, K & V from the other.

Decoder's second attention step (encoder–decoder models)



Multi-Head Attention

HOW MANY views at once

Several attention heads run in parallel — each can be self- or cross-attention — capturing different relationships.

A multiplier applied to self- or cross-attention, everywhere



Flash Attention

HOW it's computed

The exact same attention math, computed with a hardware-efficient algorithm that avoids wasted memory traffic.

Modern training & inference, for speed and longer context

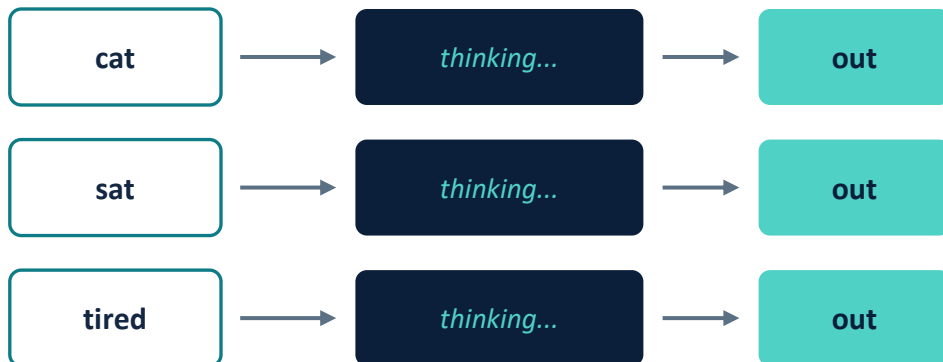
Feed-Forward, Residuals & Normalization

After attention mixes information between words, two more ingredients make the stack trainable and deep.



Feed-Forward Network

After words share context via attention, each word passes through its own small neural network — independently — to further process what it just learned.



Same tiny network, applied to every word position — in parallel.



Residual Connections + LayerNorm

Stacking dozens of layers can make training unstable. Two safety nets fix this:

- **Residual (skip) connections**

Each layer adds its output to its input, instead of replacing it — like a highway shortcut so information never gets lost.

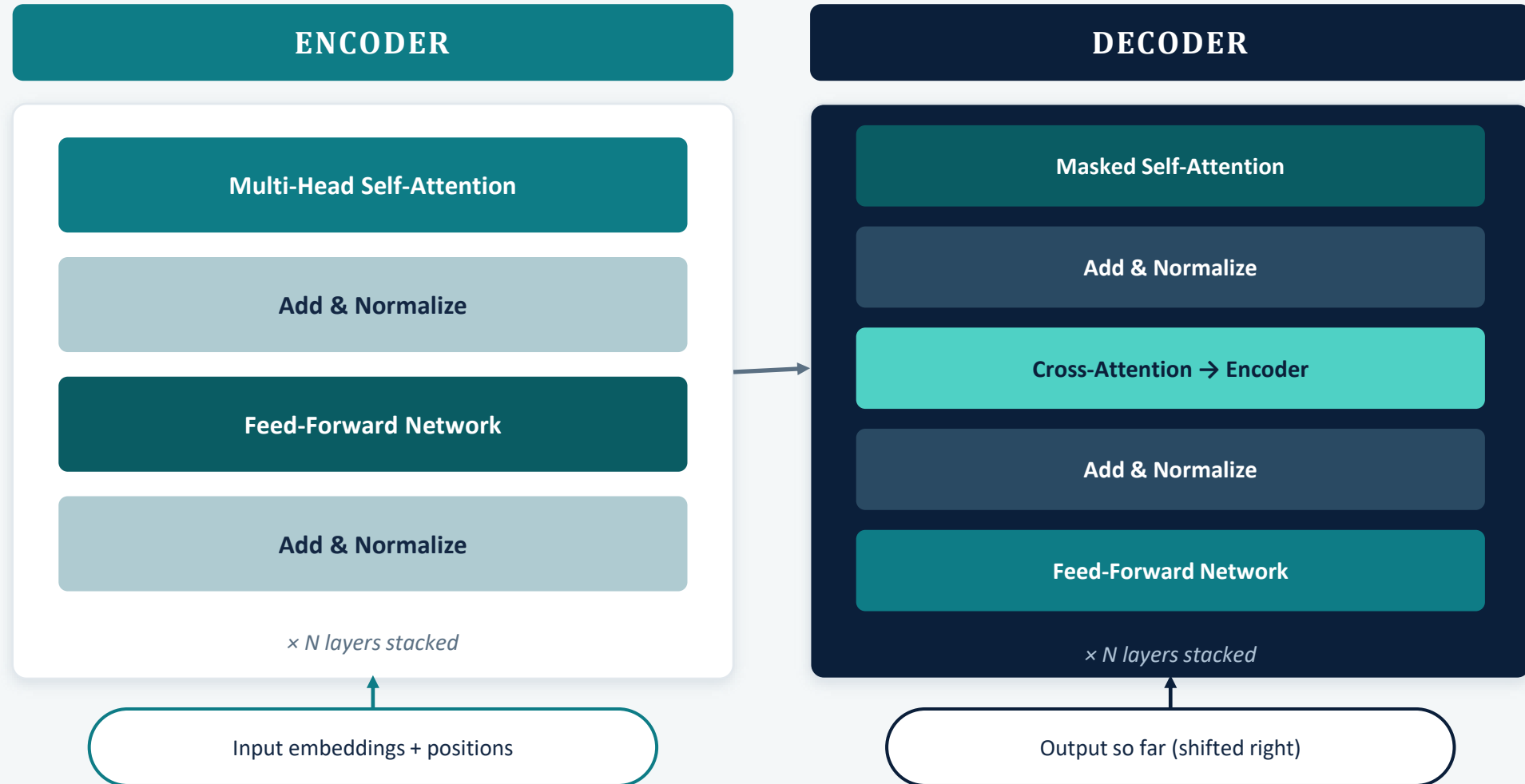
- **Layer Normalization**

Rescales the numbers after each step so values stay in a healthy range as they flow through many layers.

Result: networks with dozens of layers train smoothly instead of breaking down.

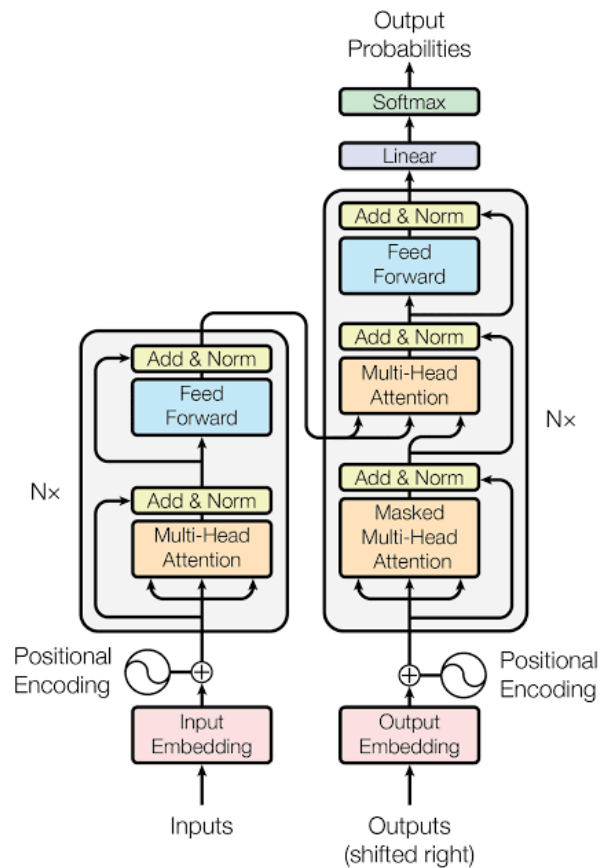
Putting It All Together

Encoder (left) reads the input. Decoder (right) generates the output, one word at a time, while looking back at the encoder.



The Original Transformer Diagram

The exact figure from the original paper — everything we just walked through, in the canonical form you'll see referenced everywhere.



Reading the Diagram

- Embedding layers**
Turn tokens into vectors (input and output)
- Multi-Head Attention**
Including the masked version in the decoder
- Add & Norm**
Residual connection plus layer normalization
- Feed Forward**
Per-word non-linear transformation
- Linear**
Projects decoder output to vocabulary size
- Softmax**
Converts final scores into output probabilities

Nx means the whole block repeats N times before moving to the next stage.

One Blueprint, Three Variants

Take the full architecture and keep only the parts each job needs.

Encoder-only

KEEP

Bidirectional self-attention

REMOVE

Decoder, cross-attention

BEST FOR

Needs to fully understand text, not generate it

EXAMPLES

BERT, RoBERTa

Decoder-only

KEEP

Masked self-attention

REMOVE

Encoder, cross-attention

BEST FOR

Needs to generate text one word at a time

EXAMPLES

GPT family

Encoder-Decoder

KEEP

Both stacks + cross-attention

REMOVE

Nothing

BEST FOR

Needs to transform one sequence into another

EXAMPLES

T5, BART

Encoder-Only: Understanding Text



Reads in Both Directions

An encoder-only model sees the entire sentence at once — left context and right context together — which makes it excellent at understanding meaning.



context flows both ways

Classification head →

Positive (98%)

A small extra layer on top turns the encoder's understanding into a label.

Typical Use Cases

- **Sentiment classification**

“Was this review positive or negative?”

- **Spam / topic detection**

“Is this email spam? What category?”

- **Named entity recognition**

“Which words are people, places, dates?”

- **Question answering (extractive)**

“Find the answer span in this passage.”

Example model: BERT (Bidirectional Encoder Representations from Transformers)

Decoder-Only: Generating Text



Only Looks Backward

A causal mask blocks each word from seeing anything ahead of it. The model must predict the next word using only what came before.

Prompt: "Once upon a time"

Once upon a time



there

Once upon a time there



was

Once upon a time there was



a

Each predicted word is fed back in to predict the next — one token at a time.

Typical Use Cases

- **Open-ended text generation**

"Write a short story about a lighthouse."

- **Conversational assistants**

"Answer my question in a helpful way."

- **Code generation**

"Write a Python function to sort a list."

- **Autocomplete**

"Suggest how to finish this sentence."

Example model: GPT (Generative Pre-trained Transformer)

Encoder-Decoder: Transforming Text



Read Fully, Then Generate

The encoder reads and understands the entire input first. The decoder then generates the output step by step, using cross-attention to consult the encoder's understanding.

“How are you?”

Encoder reads (English)



cross-attention

“Comment ça va?”

Decoder generates (French)

The two sides can even use different vocabularies and lengths.

Typical Use Cases

- **Machine translation**

“Translate this paragraph into French.”

- **Summarization**

“Condense this article into 3 sentences.”

- **Question answering (generative)**

“Answer this in your own words.”

- **Data-to-text generation**

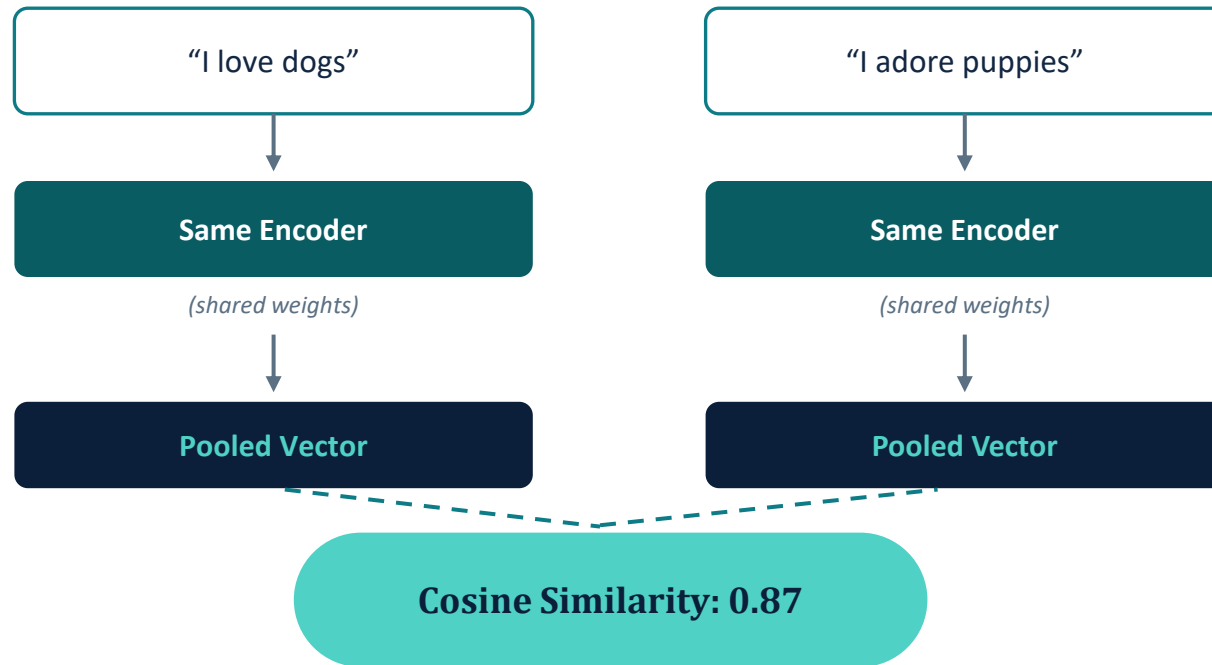
“Turn this table into a written report.”

Example models: T5, BART

Sentence Transformers: A Specialized Adaptation

Raw BERT was not specifically trained to produce sentence embeddings optimized for semantic similarity. Sentence Transformers fix this with a “twin network” design.

How It Compares Two Sentences



High similarity — the two sentences mean nearly the same thing.

Why Not Just Use BERT Directly?

Raw BERT embeddings cluster together oddly, making similarity comparisons unreliable and slow at scale.

Where This Shines

- Semantic search
- Clustering similar documents
- Duplicate / paraphrase detection
- Recommendation systems

Which Variant Should You Reach For?



Classify or label text



Encoder-only



Compare meaning / search



Sentence Transformer



Generate open-ended text



Decoder-only



Transform one sequence into another



Encoder-Decoder

Recap: One Blueprint, Many Capabilities

	Encoder-only	Decoder-only	Encoder-Decoder
Attention type	Bidirectional	Causal (masked)	Both + cross-attention
Sees future tokens?	Yes	No	Encoder: yes / Decoder: no
Typical objective	Masked word prediction	Next-word prediction	Sequence-to-sequence
Great at	Understanding & classifying	Generating fluent text	Transforming text
Example models	BERT, RoBERTa	GPT family	T5, BART

The same self-attention building block, arranged differently, produces fundamentally different capabilities.

Additional Readings

Go deeper into the concepts covered in this session



Vaswani et al., “Attention Is All You Need”

The original Transformer paper — source of the architecture diagram



Hugging Face Transformers Documentation

Architecture overview and model implementations



Sentence Transformers Documentation

Sentence embeddings, pooling, and similarity



Encoder–Decoder Transformers Documentation

Sequence-to-sequence task guides

DEMO 2

Applications of Transformers

Sentiment · QA · NER · Summarization · Translation · Vision — a live notebook walkthrough

Encoder-only

Decoder-only

Encoder-Decoder

Sentence Transf.

Vision Transf.

SUMMARY

What You Learned in Part 2

From word vectors to real Transformer applications.

Embeddings

- One-hot vs dense vectors
- Word2Vec: CBOW & Skip-gram
- GloVe & FastText
- Static vs contextual

Transformer Architecture

- Self-attention & Q/K/V
- Multi-head attention
- Positional encoding
- Encoder / Decoder / Enc-Dec

Applications

- Classification, QA, NER
- Text generation
- Summarization & translation
- Vision Transformers

GO FURTHER

Resources & Next Steps

Where to deepen what we covered today.

Word2Vec / GloVe Papers

Mikolov et al. (2013) and Pennington et al. (2014) — the original embedding papers

Gensim Documentation

Train and explore Word2Vec, FastText & GloVe models hands-on

HuggingFace Course

Free, practical introduction to Transformers — huggingface.co/course

Sentence-Transformers Docs

Build semantic search & similarity systems on top of Transformers

Thank You